Please don't forget the *dmesg* command! After a reboot, you can see the initialisation of internal sensors and devices. For example, I could clearly see that the ROM is a NAND device: *Manufacturer ID: 0x98, Chip ID: 0xbc (Toshiba NAND 512MiB 1,8 V 16-bit)* and also view information on the GPS sensor:

```
GPS: mt3326_gps_power: Switching GPS device on
GPS: mt3326_gps_power: ignore power control: 1
GPS: mt3326_gps_probe: Registering chardev
GPS: mt3326_gps_probe: major: 229, minor: 0
GPS: mt3326_gps_probe: Done
```

SSH access over Wi-Fi

For quicker shell access (and easy file transfers using the *sshfs* facility), you can install an SSH daemon and connect via Wi-Fi. I installed *QuickSSHD*, which listens on port 2222. Once installed, run the *ssh* client on your PC and connect to the IP address determined earlier via *adb*, as follows:

```
ssh -C -p 2222 10.10.1.22
QuickSSHD for Android
Single-User mode (Any username works).
Root your device to allow users.
anton@10.10.1.22's password:
$
```

Rooting the device

Basically, most applications can be happily run with ordinary user rights, but the most interesting things are available only to the *root* user. To gain *root* access, install the *z4root* and *Superuser* applications. The first app allows you to do temporary rooting, constant rooting, or to return from a rooted state to the original condition. The second app controls access to the *root* level, like the *su* Linux command on a PC. It's amazing how fast an Android phone can be rebooted by *root* :-)

With super-user privileges, you can do a lot of exciting things, including gain access to restricted parts of Android, format the SD card, change the IMEI, make a phone call via the *service* command, and much more. For example, to make a call from the first SIM card, run *service call phone 2 s16 "MyNumber"*, where *MyNumber* is the number to be dialled. Root privileges are also required if you decide to assign an internal IP address when establishing a network connection via USB cable [see Reference 5].

Building a simple console application

As you might know, the core of Android is Dalvik—a slightly modified JVM (Java Virtual Machine) intended to effectively run mobile-sphere applications. Dalvik does not align to Java SE nor Java ME class library profiles (for example, Java ME classes, AWT or Swing are not supported). Instead, it uses its own library, built on a subset of the Apache Harmony Java implementation. Let's now build a simple 'Hello, World' console application for Android [see Reference 6]. To compile the Java source on your PC, you should also have the JDK packages pre-installed. The elementary code is given below:

```
package org.apache;
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

Compile it, and package it into a *jar* archive, as follows:

```
$ javac -d . -g Helloworld.java
$ jar -cvf  Temp.jar *
added manifest
adding: classes.dex(in = 776) (out= 432)(deflated 44%)
adding: CmdLine.jar(in = 552) (out= 533)(deflated 3%)
adding: HelloWorld.java(in = 144) (out= 117)(deflated 18%)
adding: org/(in = 0) (out= 0)(stored 0%)
adding: org/apache/(in = 0) (out= 0)(stored 0%)
adding: org/apache/HelloWorld.class(in = 556) (out= 341)(deflated 38%)
```

Next, transform it into a modified archive that can be run by the Dalvik VM:

```
$ ./platform-tools/dx --dex --output=classes.dex Temp.jar
$ ./platform-tools/aapt add CmdLine.jar classes.dex
./platform-tools/aapt: /lib/libz.so.1: no version information
available (required by ./platform-tools/aapt)
 'classes.dex' as 'classes.dex'...
```

After this, transfer the final *CmdLine.jar* file to the smartphone. You can launch it as follows:

```
$ dalvikvm -cp CmdLine.jar  org.apache.HelloWorld
Hello World!
```

Ruby on Android

In addition to the usual Java applications, it's also possible to run an Android port of Ruby. While for GUI programs you must spend more time installing additional libraries, for basic tasks, the core library is already there:

```
$ dalvikvm -classpath IRB-0.5.2-release.apk org.jruby.Main -e
"puts 'Hello, this is Ruby!'"
warning: could not compile; pass -d or -J-Djruby.jit.logging.
verbose=true for more details
Hello, this is Ruby!
```